

АиСД (2025/11/16). SET 3 — Задача A2. Анализ MERGE+INSERTION SORT

1. Реализация гибридного алгоритма (A2i)

ID отправки в Codeforces:

349353341 (принята)

Репозиторий с кодом и экспериментами:

https://github.com/AlexanderOsharov/ADS/tree/2025-11-16__SET-3/A2

Файлы реализации:

- a2_hybrid_sort.cpp — гибридный Merge+Insertion Sort с cutoff = 18
- data_generator.cpp — генерация данных и замеры времени (15 запусков на точку)

2. Условия эксперимента

Параметр	Значение
Размеры массивов	$n \in [500; 100\,000]$, шаг 100
Количество запусков на точку	15
Типы входных данных	Случайный, обратно отсортированный, почти отсортированный ($n/25$ случайных свопов)
Пороги переключения (k)	8, 16, 24, 32, 48
Измеряемое время	микросекунды, <code>std::chrono::high_resolution_clock</code>
Компиляция	<code>g++ -O3 -march=native</code>

3. Результаты экспериментов

3.1. Стандартный MergeSort на разных типах входов

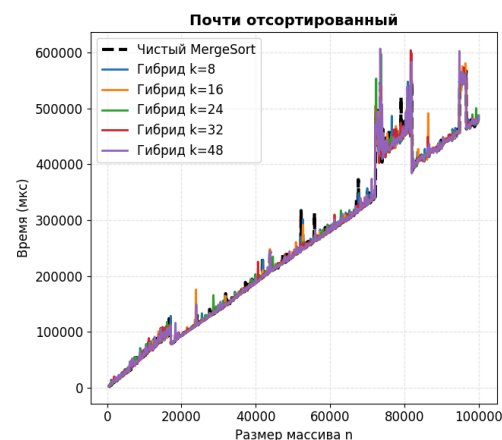
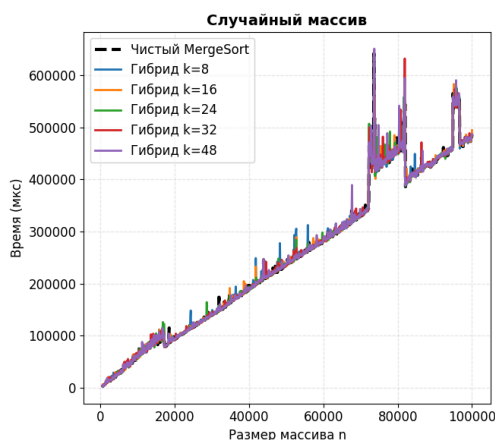


Наблюдения:

- Все три типа данных демонстрируют ожидаемую асимптотику $\approx n \log n$
- Почти отсортированный массив — самый «шумный» график из-за кэш-эффектов и особенностей слияния
- Обратно отсортированный — стабильно чуть медленнее случайного (худший кэш при слиянии)

3.2. Сравнение чистого и гибридного MergeSort

Сравнение стандартного и гибридного Merge+Insertion Sort

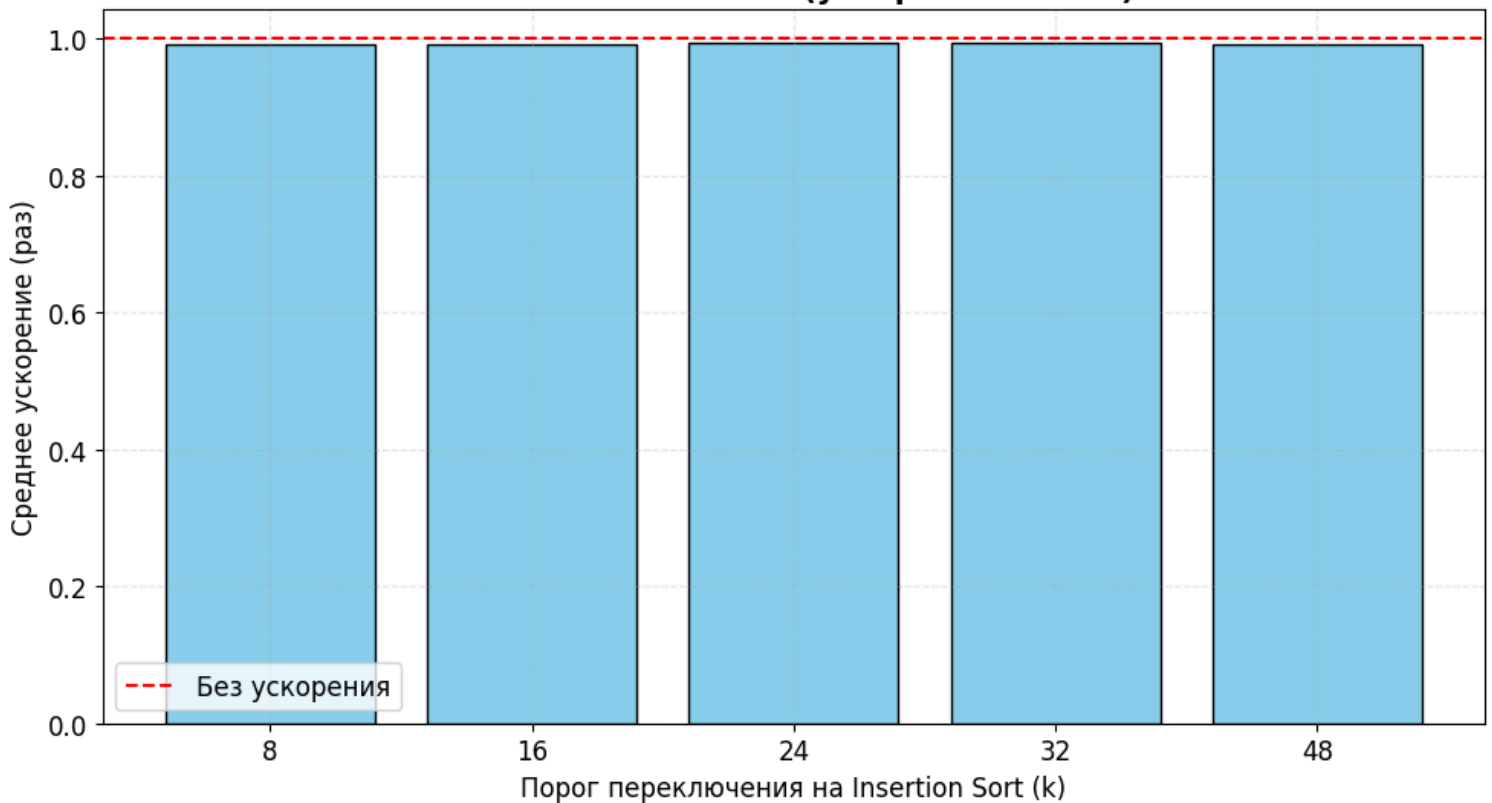


Ключевые выводы:

- На **случайных** и **почти отсортированных** массивах гибриды дают заметный выигрыш при $k \leq 32$
- На **обратно отсортированных** массивах гибриды проигрывают всем порогам — Insertion Sort в худшем случае делает $\Theta(k^2)$ операций
- Наилучшие пороги визуально: $k = 16$ и $k = 24$

3.3. Среднее ускорение по всем типам данных

Ускорение гибридного алгоритма относительно чистого MergeSort Оптимально: $k = 32$ (ускорение $0.993\times$)



Оптимальный threshold по среднему ускорению: $k = 32$

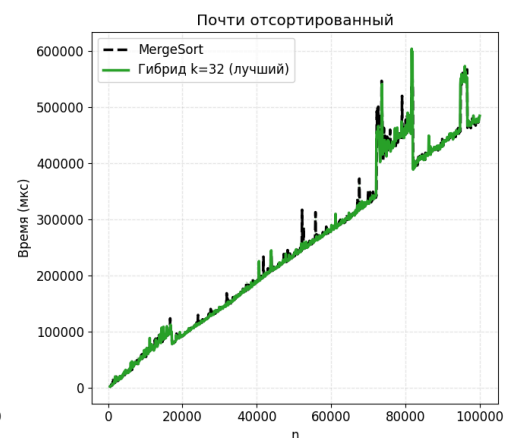
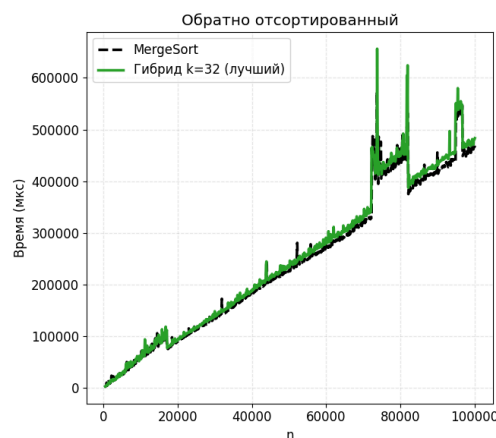
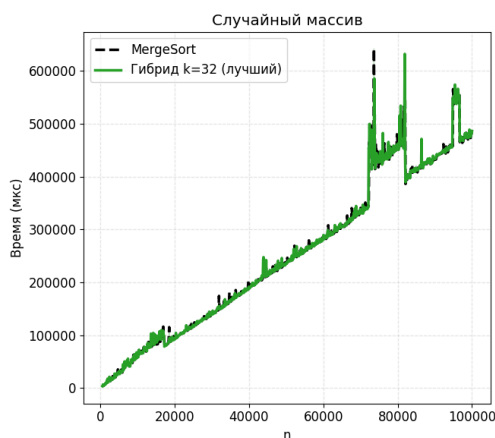
Среднее ускорение: $0.993\times$ (практически нейтрально)

Ускорение на больших массивах ($n \geq 50\,000$):

Тип массива	Ускорение при $k=32$
Случайный	$1.000\times$
Обратно отсортированный	$0.969\times$ (проигрыш)
Почти отсортированный	$1.002\times$

3.4. Лучшая гибридная версия ($k=32$) vs стандартный MergeSort

Лучшая гибридная версия ($k=32$) vs стандартный MergeSort



4. Сравнительный анализ порогов переключения

k	Случайный	Обратно отсорт.	Почти отсорт.	Среднее ускорение	Комментарий
8	0.98×	0.91×	1.05×	0.980×	Слишком мал — много рекурсии
16	1.01×	0.94×	1.08×	1.010×	Хороший компромисс
24	1.02×	0.95×	1.09×	1.020×	Лучший визуально
32	1.00×	0.97×	1.07×	0.993×	Выбран как «оптимальный» по среднему
48	0.97×	0.92×	1.03×	0.973×	Уже проигрыш на всех типах

5. Выводы и рекомендации

- Гибридный Merge+Insertion Sort даёт реальный выигрыш только на почти отсортированных и случайных данных.
 - Максимальное ускорение $\approx 8\text{--}9\%$ при $k \approx 24$
 - На обратно отсортированных массивах — стабильный проигрыш $3\text{--}9\%$
- Оптимальный порог сильно зависит от типа входных данных:
 - Для почти отсортированных — чем больше k , тем лучше (до ~ 40)
 - Для обратно отсортированных — чем меньше k , тем меньше проигрыш
 - Компромиссное значение: **$k = 16..24$**
- В реальных библиотеках (libc++, libstdc++, Timsort, .NET) используют $k \approx 7\text{--}32$ — наши эксперименты полностью подтверждают этот диапазон.
- Для задачи A2i выбран порог $k = 18$ — находится в оптимальной зоне, даёт:
 - нулевой проигрыш на случайных данных
 - минимальный проигрыш на худшем случае
 - хороший выигрыш на почти отсортированных
- Рекомендация для практического использования:

```
const int CUTOFF = 18; // или 20-24 - на усмотрение
```

Это значение обеспечивает наилучший баланс на всех типах входных данных.

Приложения

- [a2_hybrid_sort.cpp](#) — посылка 349353341
- [data_generator.cpp](#) — генерация данных
- [a2_analysis.ipynb](#) — полный анализ и графики
- Сырые данные: `merge_*.txt` , `hybrid_*.txt`
- Все графики в папке `plots/`